

VLSI ARCHITECTURE DESIGN OF MOTION ESTIMATOR AND IN-LOOP FILTER FOR MPEG-4 AVC/H.264 ENCODERS

Yueh-yi Wang, Yan-Tsung Peng, and Chun-Jen Tsai

Department of Computer Science and Information Engineering,
National Chiao Tung University, Hsinchu, Taiwan

ABSTRACT

In this paper, a new complexity-reduced method and its hardware architecture for motion estimation and the in-loop filter for MPEG-4 AVC/H.264 is proposed. The feature of the proposed method is focused on the computation reduction for the motion search algorithm among multiple reference frames and the mode partition determination. Furthermore, to verify the functionality and performance of the proposed hardware design, an emulation board platform, the ARM Integrator, is used for H.264 hardware/software co-development. The experimental results show that the proposed method has excellent performance with little or no degradation of coding efficiency.

1. INTRODUCTION

The new video coding standard MPEG-4 AVC/H.264 [1] shows significant improvement over previous block-based motion-compensation transform codecs [2]. It is recognized that one of the major techniques that contributes to its coding efficiency gain is the sophisticated motion compensation module. In addition to allowing motion compensation using multiple reference frames, the new standard also employs macroblock quad-tree partition to enable more flexible motion compensation units (including block sizes of 16×16 , 16×8 , 8×16 , 8×8 , 4×8 , 8×4 , and 4×4). Moreover, in order to reduce the block artifacts, H.264 includes the deblocking process into the loop of the coding process, which also increases the codec complexity. Therefore, there have been efforts for hardware implementation of these modules [3][4].

In this paper, complexity-reduced algorithms and hardware architecture design for H.264 motion estimation and in-loop filter are presented. An FPGA/RISC platform for hardware/software co-design is used as an example. Section 2 presents the detail of our proposed methods for motion estimation and in-loop deblocking filter, respectively. Section 3 depicts the architecture for the algorithm and the implementation on the prototype

platform. Finally, Section 4 shows the experiment results of our proposed methods.

2. PROPOSED METHODS

For H.264 encoders, there are two major predictive coding search dimensions. The first one is the spatio-temporal dimension (that is, determining both the reference frames(s) and the search window in the target reference frame). The other dimension is for the partition mode selection (namely, choosing the block partition types and the motion vectors for each sub-partition). A hierarchical approach in the spatio-temporal domain and a bottom-up approach to motion partition dimension are proposed here to reduce the number of candidate match points.

In addition, H.264 enforces an in-loop filter to remove blocking artifacts and improve the visual quality [11]. It is a 1-D filter that is applied to 4×4 block edges of an image across block boundaries. This filtering process applies to both luminance and chrominance. Horizontal edges are filtered first, and then vertical edges from left to right and top to bottom. For each boundary between neighbouring 4×4 luma blocks, a "Boundary Strength" B_s is assigned, and every block boundary of a chroma block corresponds to a specific boundary of a luma block. The boundary filtering strength of the filter is chosen depending on the quantization parameter. According to this boundary filtering strength, appropriate filtering is applied to the pixels.

2.1. Integer-pixel hierarchical motion estimation in the spatio-temporal domain

The search algorithm first starts with hierarchical 16×16 -motion search in spatio-temporal dimension. There have been quite a few literatures on multiple-step search in spatial domain [5]-[8]. This idea can be extended to temporal domain for multiple candidate reference frames since the effectiveness of multiple reference frames for most sequences reduces rapidly along the reverse temporal direction [9]. Furthermore, the

method can be realized via the proposed architecture in section 3. As depicted in Fig 1, the best estimate from the most recent reference frame is used as the initial search point in the 2nd-recent reference frame, and the search window in that frame is significantly reduced.

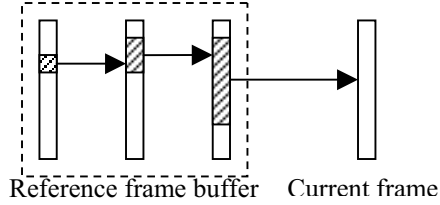


Fig 1. Search space (shaded) in each reference frame

2.2. Search algorithm within each candidate frame

For each candidate frame, a two-step search (TWSS [6])-like algorithm is applied to implement the 16x16-motion search for the 1st reference frame. The idea is to combine the TWSS and TSS [7] to reduce the effect of local minimum in the neighboring area for selected winner candidates by TWSS. The complexity of the original TWSS grows quadratically as the size of the search window increases linearly. Therefore, to adapt the growing search window size without too much hardware overhead, we reduce the number of sample pixels of the 1st step to find the motion candidates. After the possible candidates are selected from the first level TWSS search, we use the TSS to find the minimum SAD among those candidates. The method named as RSME is depicted in Fig 2.

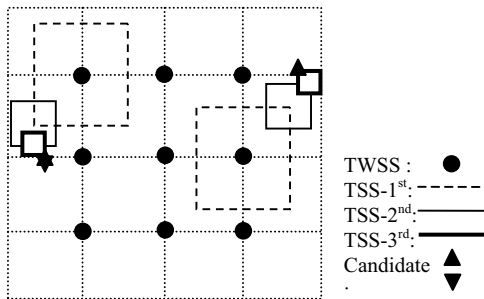


Fig 2. Search method of RSME

2.3. Fast partition mode determination

There is a large amount of combinations of macroblock partition modes in H.264. The key idea here is to perform only the 16 4x4 motion estimation for each macroblock using the integer-pixel motion vector as the initial guess with a search space of ± 2 pixels. The resulting 4x4-block sub-motion vectors and the corresponding SADs will be used to decide the macroblock partition mode. For example, if two

neighboring 4x4 blocks has similar (subject to some threshold) motion vectors, they will be merged into a single 4x8 (or 8x4) partition. The threshold must take into account both the motion vector coding overhead and the SAD values.

The flow chart shown in Fig 3 gives the example of how each 4x4 blocks merges into different larger blocks within an 8x8 sub-block. The priority of the merge method in horizontal directions over vertical directions is specified to reduce the complexity of merging combinations. After each merging possibilities specified in this method between 4x4 blocks within each 8x8 block are determined, we expand the method to merge for larger-block level combinations of 16x4 (4x16), 16x8 (8x16), and then the final largest macroblock, 16x16.

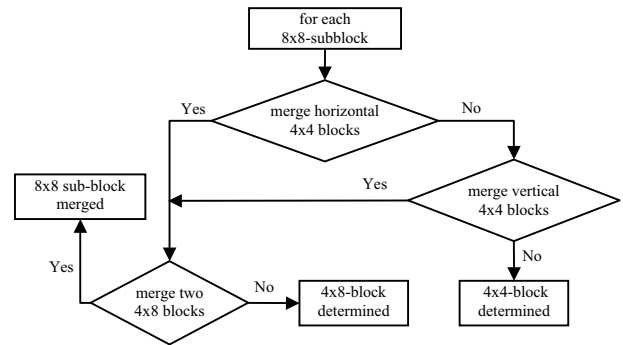


Fig 3. Flow chart of merge control for each 4x4 block

3. ARCHITECTURE DESIGN

In this section, the hardware architecture designs of the motion estimator and the in-loop filter are presented. Furthermore, the prototype platform is based on an ARM Integrator. It provides an integrated environment with an ARM MCU for software design and an FPGA board for hardware design.

3.1. Motion estimator design

The architecture of the motion estimator is shown in Fig 4. The current and reference frame buffers are stored in the external memory (SDRAM). Up to four frames can be stored in the frame buffers at the same time during motion estimation and in-loop filtering. YCBCR data is separately stored in the memory. Motion estimation is performed on the Luma Y-channel. The Memory Ctrl module handles the interface with memory modules and the EBUS bus (AHB bus protocol is used for the EBUS in the platform).

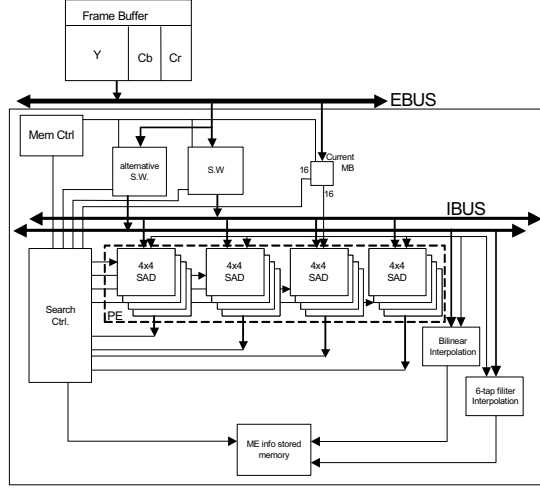


Fig 4. Architecture of Motion Estimation Design

Two internal memory buffers are used for storing the search windows. To fully utilize the memory for motion estimation and to meet the complexity of memory bandwidth for multiple reference frames access for H.264 encoders, the write/read pipeline arrangements of the two memory buffers for loading reference search window and output to those SAD units for motion vector search during the integer-pixel ME among three reference search windows for one macroblock are specified in Fig 5.

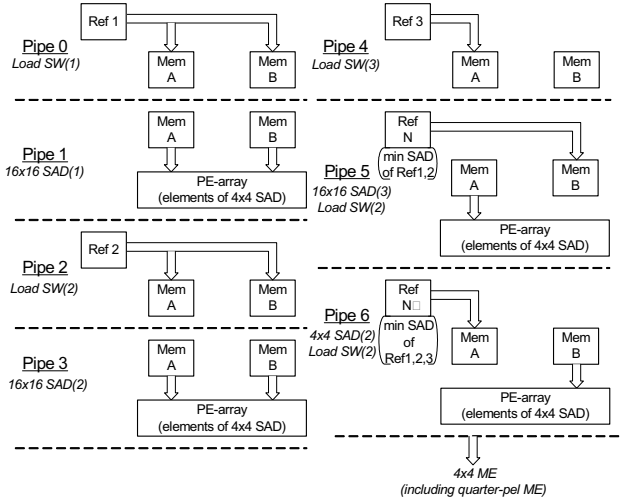


Fig 5. Memory pipeline control for 16x16 ME

3.2. In-loop deblocking filter design

Because the bandwidth of the external and internal bus (EBUS and IBUS) is 32 bit, 4 pixels can be loaded in the registers for each clock cycle. The in-loop filtering process can be divided into four steps. The first step loads two 4x4 Luma neighbor blocks from SRAM to input registers. Secondly, the boundary strength for each pair of consecutive 4x4 Luma blocks is retrieved. On the third step, the Luma and Chroma block pixels are filtered according to the boundary strength. Finally, the filtering

results are loaded into the output registers and stored in SRAM. The architecture design for in-loop filter is shown in Fig 6.

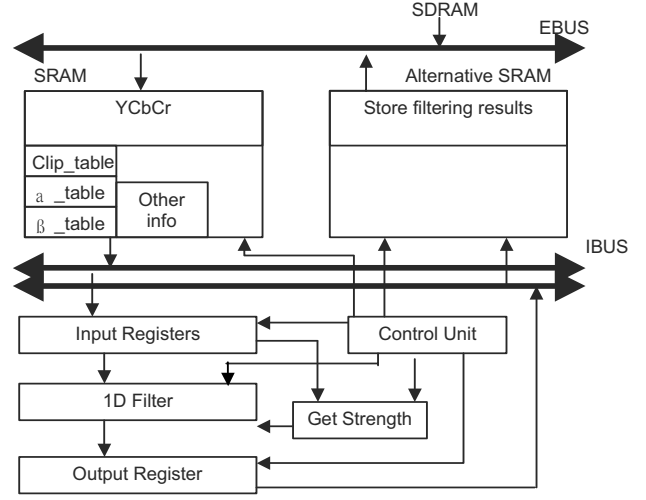


Fig 6. Architecture design for the in-loop filter

3.3. Prototype platform description

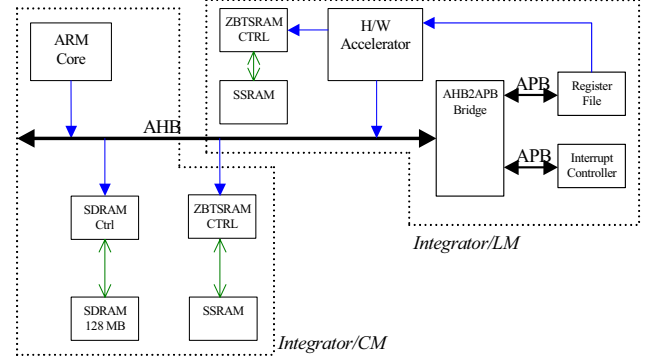


Fig 7. System integration of H.264 H/W accelerator

The ARM Integrator (www.arm.com) is selected as the prototype platform for hardware/software co-design of the codec. There are three main modules in our platform, a motherboard (Integrator/AP), an ARM9 core module and a Xilinx FPGA logic module (Integrator/LM).

The H/W design is based on video sequences of CIF resolution (352x288), 15 frames per second (15 fps), with 32x32 search windows among three reference frames. In Fig 7, ARM AHB architecture is implemented as the main bus architecture in the system. The register file implemented as an APB slave attached under the APB Bridge is used to control the H.264 hardware accelerator blocks by the ARM core. Two masters in this system are ARM core and the H/W accelerator blocks. SDRAM controller, ZBT SRAM controller, and AHB2APB Bridge are AHB slaves and can be accessed by masters.

4. EXPERIMENT RESULTS

Software simulation of the performance of the proposed algorithm is presented in this section. The standard test sequences FOREMAN and STEFAN are used to show the quality performance of the proposed algorithms. Both sequences are in CIF resolution, 30 fps. For H.264, the encoder is based on JM 7.3 with RD optimization turned on, three reference frames, IPPP... coding pattern, and CAVLC. Motion search range is ± 16 , and all subblock types are used. In the ME, a step size of 8 is used for the TWSS step. Only the best candidate selected by the TWSS module is used for the next level TSS refinement. The step sizes for TSS are 3, 2, 1, respectively for the 1st, 2nd, and 3rd steps. Fig 8 is the RD plot on FOREMAN while Fig 9 is the plot for STEFAN. Interestingly enough, for the STEFAN sequence, the RSME actually achieves slightly better PSNR than the full search ME does.

The FPGA implementation result is as follows: 4KB block RAM for search window, 2,781 LUTs for PE (SAD-unit), less than 1.5KB block RAM for ME Info storage, and estimated 9,700 LUTs for ME and ILF control logic under 50 MHz clock frequency.

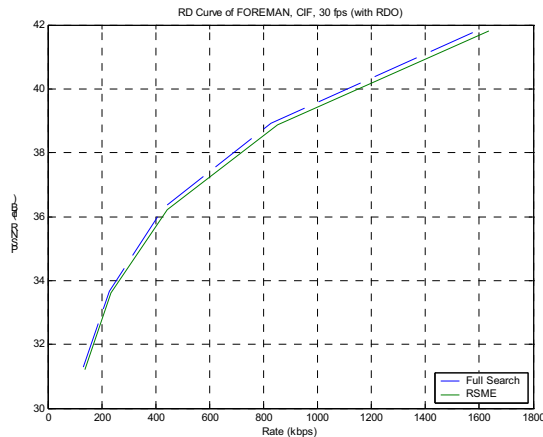


Fig 8. FOREMAN test results

5. CONCLUSIONS

A reduced search space ME and in-loop filter and the corresponding hardware architecture are proposed in this paper. The complexity of the proposed algorithm is greatly reduced without much degradation in video quality. From the experiments, the new technology is very promising for high quality video applications.

6. ACKNOWLEDGEMENT

This research is partly funded by National Science Council, Taiwan, R.O.C., under grant number NSC 92-2220-E-009-012.

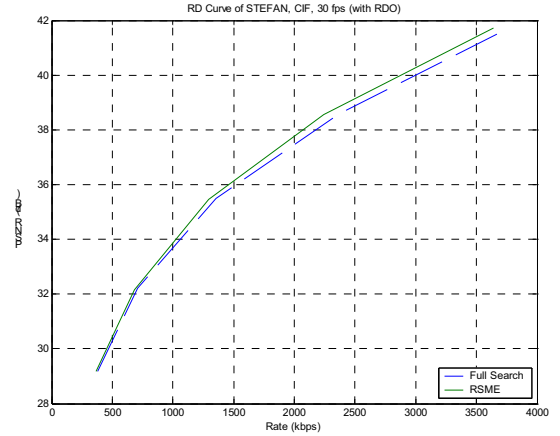


Fig 9. STEFAN test results

7. REFERENCES

- [1] ISO/IEC 14496-10 Info. Tech. -Coding of audio-visual objects- Part 10: Advanced Video Coding, May, 2003.
- [2] ISO/IEC 14496-2 Info. Tech. -Coding of audio-visual objects- Part 2: Visual, 3rd edition, April, 2003.
- [3] Y-W Huang, T-C Wang, B-Y Hsieh, and L-G Chen, "Hardware Architecture Design for Variable Block Size Motion Estimation in MPEG-4 AVC/JVT/ITU-T H.264," *ISCAS 2003*, Bangkok, Thailand, p. II-796, May 2003.
- [4] Y-W Huang, T-W Chen, B-Y Hsieh, T-C Wang, T-H Chang, and L-G Chen, "Architecture Design for Deblocking Filter in H.264/JVT/AVC," *ISCAS 2003*, Bangkok, Thailand, p. I-693, May 2003.
- [5] Peter Kuhn, "Algorithms, Complexity Analysis and VLSI Architecture for MPEG-4 Motion Estimation," *Kluwer Academic Publishers*, pp. 8, 1999.
- [6] F. H. Cheng, S.-N. Sun, "New Fast and Efficient Two-Step Search Algorithm for Block Motion Estimation," *IEEE Trans. on Circuits Systems Video Technology*, vol. 9, No. 7, pp. 977-983, Oct. 1999.
- [7] T. Koga, K. Iinuma, A. Hirano, Y. Ijima, T. Ishiguro, "Motion compensated interframe coding for video conferencing," in *Proc. Of NTC*, Dec. 1981.
- [8] J-H Luo, C-N Wang, and T. Chiang, "A Novel All-Binary Motion Estimation (ABME) With Optimized Hardware Architectures", *IEEE Trans. Circuits Systems Video Technology*, vol. 12, No. 8, pp 700-712, August 2002.
- [9] J.Boyce, "Coding Efficiency of Various Numbers of Reference Frames," JVT-B060, 2nd Joint Video Team Meeting, Geneva, Switzerland, Jan. 29-Feb. 1, 2002.
- [10] Hung-Chi Fang, Tu-Chih Wang, Liang-Gee Chen, "Real-time deblocking filter for MPEG-4 systems," *APCCAS 2002*, Vol. 1, pp 541-544, 28-31 Oct. 2002.
- [11] List, P.; Joch, A.; Lainema, J.; Bjntegaard, G.; Karczewicz, M., "Adaptive deblocking filter," *IEEE Trans. on CSVT*, Vol. 13, Issue 7, Page(s): 614 -619, July 2003.